

使用 lit-element 建構故事元件

來源：<https://codelabs.developers.google.com/codelabs/lit-story-viewer?hl=zh-tw>

步驟數：8

在這個程式碼研究室中，您將使用 LitElement 建構故事網頁元件。

目錄

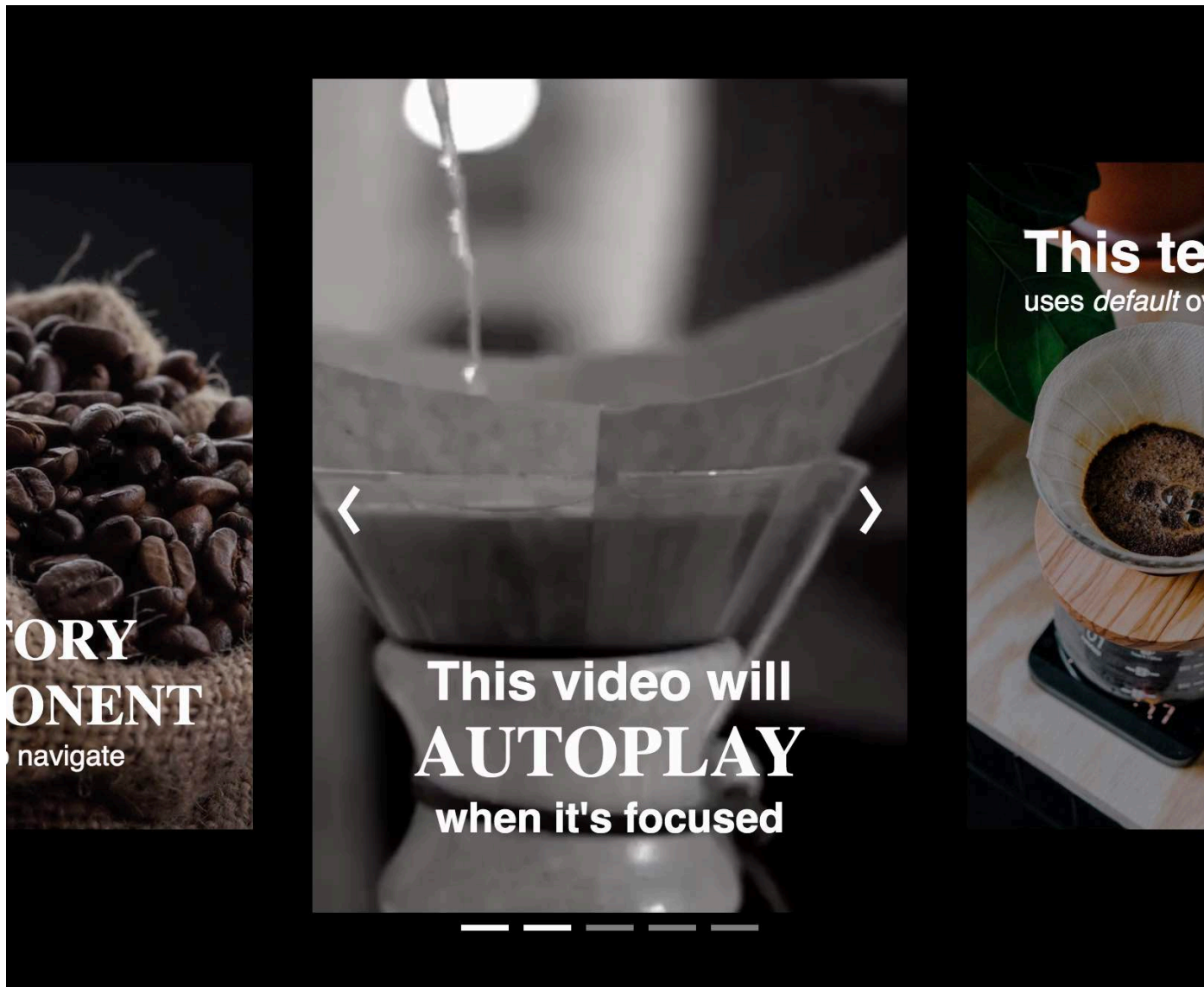
1. [1. 簡介](#)
2. [2. 設定](#)
3. [3. <story-card> 元件](#)
4. [4. <story-viewer> 元件](#)
5. [5. 進度列和導覽](#)
6. [6. 滑動](#)
7. [7. 自動播放](#)
8. [8. Tip the Scales](#)

步驟 1 · 約 2 分鐘

1. 簡介

近來，動態消息是熱門的 UI 元件。社群和新聞應用程式正將這些資訊整合到動態消息中。在本程式碼研究室中，我們將使用 `lit-element` 和 `TypeScript` 建構故事元件。

故事元件的最終樣貌如下：



我們可以將社群媒體或新聞「故事」視為一系列要依序播放的資訊卡，有點像是投影片。事實上，故事就是投影片。資訊卡通常會顯示圖片或自動播放的影片，頂端可能還會顯示額外文字。以下是我們要建構的內容：

功能清單

- 以圖片或影片為背景的資訊卡。
- 向左或向右滑動即可瀏覽故事。
- 自動播放影片。
- 可新增文字或以其他方式自訂資訊卡。

就這個元件的開發人員體驗而言，最好能以純 HTML 標記指定動態消息資訊卡，如下所示：

```
<story-viewer>
  <story-card>
    
    <h1>Title</h1>
  </story-card>
  <story-card>
    <video slot="media" src="some/video.mp4" loop playsinline></video>
    <h1>Whatever</h1>
    <p>I want!</p>
  </story-card>
</story-viewer>
```

因此，我們也將這項功能加入功能清單。

功能清單

- 在 HTML 標記中接受一系列的資訊卡。

這樣一來，只要撰寫 HTML，任何人都能使用我們的故事元件。無論您是否為程式設計師，都可以使用這項功能，而且適用於所有支援 HTML 的位置，例如內容管理系統、架構等。

必要條件

- 可執行 `git` 和 `npm` 的殼層
 - 文字編輯器
-

步驟 2 · 約 2 分鐘

2. 設定

首先，請複製這個存放區：[story-viewer-starter](#)



```
git clone git@github.com:PolymerLabs/story-viewer-starter.git
```

環境已設定 `lit-element` 和 `TypeScript`。只要安裝依附元件即可：



```
npm i
```

如果是 VS Code 使用者，請安裝 [lit-plugin](#) 擴充功能，取得 `lit-html` 範本的自動完成、型別檢查和 Linting 功能。

執行下列指令，啟動開發環境：

```
npm run dev
```

您現在可以開始寫程式了！

3. <story-card> 元件

建構複合元件時，有時從較簡單的子元件開始，然後逐步建構，會比較容易。因此，我們先來建立 `<story-card>`。應能顯示全出血影片或圖片。使用者應可進一步自訂，例如疊加文字。

第一步是定義擴充 `LitElement` 的元件類別。`customElement` 裝飾器會負責為我們註冊自訂元素。現在是確保你在 [tsconfig 中啟用裝飾器](#) 的好時機，請使用 `experimentalDecorators` 旗標 (如果你使用入門存放區，裝飾器已啟用)。

將下列程式碼放入 `story-card.ts`：

```
import { LitElement } from 'lit';
import { customElement } from 'lit/decorators.js';

@customElement('story-card')
export class StoryCard extends LitElement {
}
```

現在 `<story-card>` 是可用的自訂元素，但目前沒有任何內容可顯示。如要定義元素的內部結構，請定義 `render` 執行個體方法。我們將使用 `lit-html` 的 `html` 標記，在此提供元素的範本。

這個元件的範本應包含哪些內容？使用者應能提供兩項內容：媒體元素和重疊顯示的圖片。因此，我們將為每個項目新增一個 `<slot>`。

我們可透過「插槽」指定應如何算繪自訂元素的子項。如需更多資訊，請參閱[這篇逐步導覽文章](#)，瞭解如何使用 Slot。

```
import { html } from 'lit';

export class StoryCard extends LitElement {
  render() {
    return html`
      <div id="media">
        <slot name="media"></slot>
      </div>
      <div id="content">
        <slot></slot>
      </div>
    `;
  }
}
```

將媒體元素放入專屬版位，有助於我們以該元素為目標，新增全版面樣式和自動播放影片等。將第二個 slot (自訂疊加層的 slot) 放在容器元素內，以便稍後提供一些預設邊框間距。

現在可以這樣使用 `<story-card>` 元件：

```
<story-card>
  
  <h1>My Title</h1>
  <p>my description</p>
</story-card>
```

但看起來很糟糕：



My Title

my description

新增樣式

讓我們新增一些樣式。使用 `lit-element` 時，我們定義靜態 `styles` 屬性，並傳回以 `css` 標記的範本字串，即可完成上述操作。在這裡編寫的 CSS 只會套用至自訂元素！以這種方式使用 CSS 和 shadow DOM 真的很棒。

讓我們設定已插入的媒體元素樣式，以涵蓋 `<story-card>`。在此，我們可以為第二個位置的元素提供一些美觀的格式。這樣一來，元件使用者可以放入一些 `<h1>`、`<p>` 或任何內容，並預設看到美觀的內容。

```
import { css } from 'lit';

export class StoryCard extends LitElement {
  static styles = css`
    #media {
      height: 100%;
    }
    #media ::slotted(*) {
      width: 100%;
      height: 100%;
      object-fit: cover;
    }

    /* Default styles for content */
    #content {
      position: absolute;
      top: 0;
      right: 0;
      bottom: 0;
      left: 0;
      padding: 48px;
      font-family: sans-serif;
      color: white;
      font-size: 24px;
    }
    #content > slot::slotted(*) {
      margin: 0;
    }
  `;
}
```



現在我們有了背景媒體的動態消息資訊卡，可以在頂端放置任何內容。太好了！我們稍後會回到 `StoryCard` 類別，實作自動播放影片。

4. <story-viewer> 元件

<story-viewer> 元素是 <story-card> 的父項。這個函式會負責水平排列卡片，並讓使用者在卡片間滑動。我們會以與 StoryCard 相同的方式啟動。我們要將故事卡片新增為 <story-viewer> 元素的子項，因此請為這些子項新增插槽。

將下列程式碼放入 story-viewer.ts：

```
import { LitElement, html } from 'lit';
import { customElement } from 'lit/decorators.js';

@customElement('story-viewer')
export class StoryViewer extends LitElement {
  render() {
    return html`<slot></slot>`;
  }
}
```

接著是橫向版面配置。我們可以為所有已安插的 <story-card> 提供絕對位置，並根據索引轉譯這些位置。我們可以使用 :host 選取器，指定 <story-viewer> 元素本身。

```
static styles = css`
:host {
  display: block;
  position: relative;
  /* Default size */
  width: 300px;
  height: 800px;
}
::slotted(*) {
  position: absolute;
  width: 100%;
  height: 100%;
}`;
```

使用者只要在主機上從外部覆寫預設高度和寬度，就能控制微故事資訊卡的大小。如下所示：

```
story-viewer {
  width: 400px;
  max-width: 100%;
  height: 80%;
}
```

如要追蹤目前檢視的卡片，請在 StoryViewer 類別中新增例項變數 index。使用 LitElement 的 @property 裝飾器裝飾該屬性，即可在值變更時重新轉譯元件。

```
import { property } from 'lit/decorators.js';

export class StoryViewer extends LitElement {
  @property({type: Number}) index: number = 0;
}
```

每張資訊卡都必須水平平移到定位點。我們會在 `lit-element` 的 [update 生命週期方法](#) 中套用這些翻譯。只要這個元件的觀察屬性發生變化，更新方法就會執行。通常我們會查詢該時段，並在 `slot.assignedElements()` 中進行迴圈。不過，由於我們只有一個未命名的時段，因此這與使用 `this.children` 相同。為方便起見，我們使用 `this.children`。

```
import { PropertyValues } from 'lit';

export class StoryViewer extends LitElement {
  update(changedProperties: PropertyValues) {
    const width = this.clientWidth;
    Array.from(this.children).forEach((el: Element, i) => {
      const x = (i - this.index) * width;
      (el as HTMLElement).style.transform = `translate3d(${x}px,0,0)`;
    });
    super.update(changedProperties);
  }
}
```

我們的 `<story-card>` 現在都排成一列。只要我們注意適當設定樣式，它仍可做為其他元素的子項：

```
<story-viewer>
  <!-- A regular story-card child... -->
  <story-card>
    <video slot="media" src="some/video.mp4"></video>
    <h1>This video</h1>
    <p>is so cool.</p>
  </story-card>
  <!-- ...and other elements work too! -->
  
</story-viewer>
```

前往 `build/index.html`，並取消註解其餘故事卡片元素。現在，讓我們設定導覽功能！

步驟 5 · 約 6 分鐘

5. 進度列和導覽

接著，我們要新增在卡片之間切換的方式，以及進度列。

讓我們在 `StoryViewer` 中新增一些輔助函式，以便瀏覽故事。他們會為我們設定索引，同時將索引限制在有效範圍內。

在 `story-viewer.ts` 的 `StoryViewer` 類別中，新增：

```
/** Advance to the next story card if possible */
next() {
  this.index = Math.max(0, Math.min(this.children.length - 1, this.index + 1));
}

/** Go back to the previous story card if possible */
previous() {
  this.index = Math.max(0, Math.min(this.children.length - 1, this.index - 1));
}
```

如要向使用者顯示導覽功能，請在 `<story-viewer>` 中新增「上一頁」和「下一頁」按鈕。點選任一按鈕時，我們希望呼叫 `next` 或 `previous` 輔助函式。`lit-html` 可輕鬆將事件監聽器新增至元素；我們可以同時算繪按鈕並新增點擊事件監聽器。

將 `render` 方法更新為以下內容：

```
export class StoryViewer extends LitElement {
  render() {
    return html`
      <slot></slot>

      <svg id="prev" viewBox="0 0 10 10" @click=${() => this.previous()}>
        <path d="M 6 2 L 4 5 L 6 8" stroke="#fff" fill="none" />
      </svg>
      <svg id="next" viewBox="0 0 10 10" @click=${() => this.next()}>
        <path d="M 4 2 L 6 5 L 4 8" stroke="#fff" fill="none" />
      </svg>
    `;
  }
}
```

請參閱如何直接在 `render` 方法中，為新的可擴充向量圖形按鈕加入事件監聽器。這適用於任何事件。只要將表單的繫結 `@eventname=${handler}` 新增至元素即可。

在 `static styles` 屬性中新增下列項目，即可設定按鈕樣式：

```
svg {
  position: absolute;
  top: calc(50% - 25px);
  height: 50px;
  cursor: pointer;
}
#next {
  right: 0;
}
```

進度列方面，我們會使用 CSS 格線為每個故事資訊卡設定小方塊的樣式。我們可以透過 `index` 屬性，有條件地將類別新增至方塊，指出方塊是否「已看過」。我們可以採用 `i <= this.index : 'watched': ''` 等條件運算式，但如果新增更多類別，程式碼可能會變得冗長。幸好，`lit-html` 提供名為 `classMap` 的指令，可協助解決這個問題。首先，請匯入 `classMap`：

```
import { classMap } from 'lit/directives/class-map';
```

並在 `render` 方法底部新增下列標記：

```
<div id="progress">
  ${Array.from(this.children).map((_, i) => html`
    <div
      class=${classMap({watched: i <= this.index})}
      @click=${() => this.index = i}
    ></div>`
  )}
</div>
```

我們也加入更多點擊處理常式，讓使用者可以直接跳到特定故事資訊卡。

以下是可新增至 `static styles` 的新樣式：

```
::slotted(*) {
  position: absolute;
  width: 100%;
  /* Changed this line! */
  height: calc(100% - 20px);
}

#progress {
  position: relative;
  top: calc(100% - 20px);
  height: 20px;
  width: 50%;
  margin: 0 auto;
  display: grid;
  grid-auto-flow: column;
  grid-auto-columns: 1fr;
  grid-gap: 10px;
  align-content: center;
}

#progress > div {
  background: grey;
  height: 4px;
  transition: background 0.3s linear;
  cursor: pointer;
}

#progress > div.watched {
  background: white;
}
```

導覽和進度列已完成。現在來加點創意！

6. 滑動

如要實作滑動功能，我們將使用 [Hammer.js](#) 手勢控制程式庫。Hammer 會偵測平移等特殊手勢，並傳送含有相關資訊 (例如 X 軸的變化量) 的事件，供我們使用。

以下說明如何使用 Hammer 偵測平移，並在發生平移事件時自動更新元素：

```
import { state } from 'lit/decorators.js';
import 'hammerjs';

export class StoryViewer extends LitElement {
  // Data emitted by Hammer.js
  @state() _panData: {isFinal?: boolean, deltaX?: number} = {};

  constructor() {
    super();
    this.index = 0;
    new Hammer(this).on('pan', (e: HammerInput) => this._panData = e);
  }
}
```

LitElement 類別的建構函式也是在主機元素本身附加事件監聽器的絕佳位置。Hammer 建構函式會採用要偵測手勢的元素。在本例中，就是 `StoryViewer` 本身，也就是 `this`。接著，我們使用 Hammer 的 API，告知要偵測「平移」手勢，並將平移資訊設定為新的 `_panData` 屬性。

使用 `@state` 裝飾 `_panData` 屬性後，LitElement 會觀察 `_panData` 的變更並執行更新，但屬性不會有相關聯的 HTML 屬性。

接著，請擴增 `update` 邏輯，以使用平移資料：

```

// Update is called whenever an observed property changes.
update(changedProperties: PropertyValues) {
  // deltaX is the distance of the current pan gesture.
  // isFinal is whether the pan gesture is ending.
  let { deltaX = 0, isFinal = false } = this._panData;
  // When the pan gesture finishes, navigate.
  if (!changedProperties.has('index') && isFinal) {
    deltaX > 0 ? this.previous() : this.next();
  }
  // We don't want any deltaX when releasing a pan.
  deltaX = isFinal ? 0 : deltaX;
  const width = this.clientWidth;
  Array.from(this.children).forEach((el: Element, i) => {
    // Updated this line to utilize deltaX.
    const x = (i - this.index) * width + deltaX;
    (el as HTMLElement).style.transform = `translate3d(${x}px,0,0)`;
  });

  // Don't forget to call super!
  super.update(changedProperties);
}

```

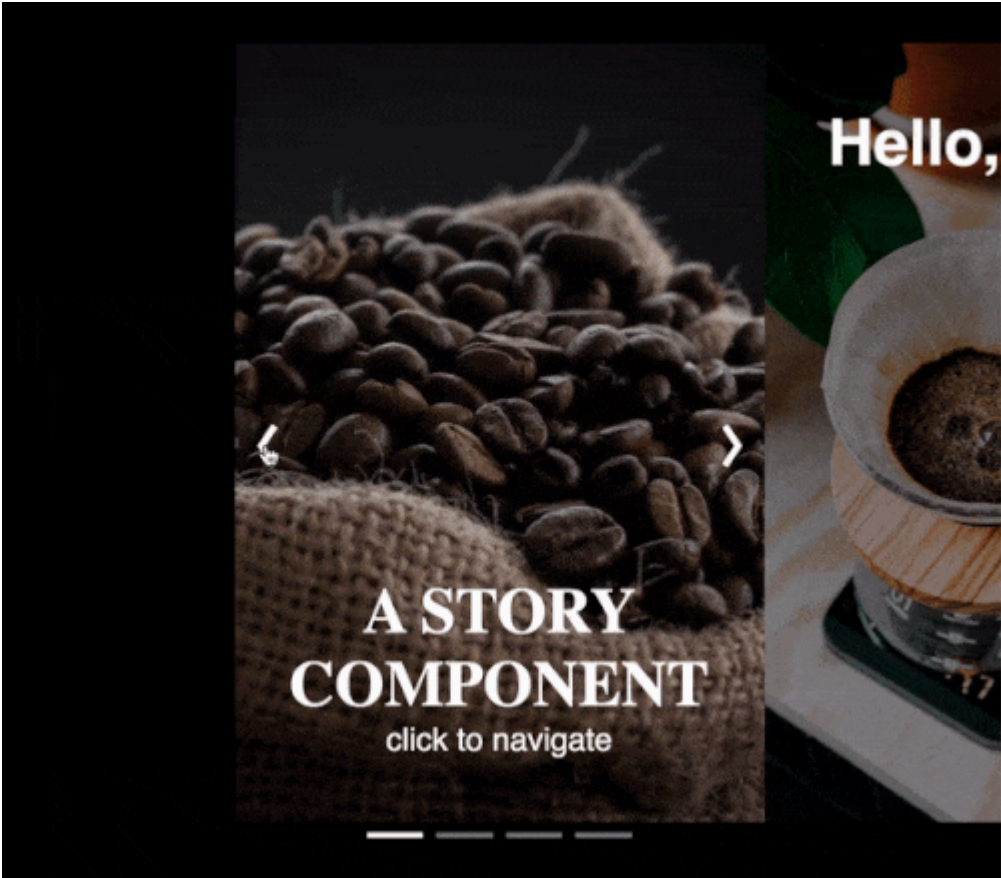
現在可以來回拖曳故事卡片。為了讓一切順利進行，請返回 `static get styles` 並將 `transition: transform 0.35s ease-out`; 新增至 `::slotted(*)` 選取器：

```

::slotted(*) {
  ...
  transition: transform 0.35s ease-out;
}

```

現在滑動操作更順暢了：



7. 自動播放

最後要新增的功能是自動播放影片。當故事資訊卡進入焦點時，我們希望背景影片 (如有) 能開始播放。當故事資訊卡離開焦點時，我們應暫停播放影片。

我們會透過在索引變更時，對適當子項傳送「entered」和「exited」自訂事件，來實作這項功能。在 `StoryCard` 中，我們會收到這些事件，並播放或暫停任何現有影片。為什麼選擇在子項上調度事件，而不是呼叫 `StoryCard` 上定義的「entered」和「exited」例項方法？如果使用方法，元件使用者就只能編寫自訂元素，才能使用自訂動畫編寫自己的故事資訊卡。使用事件時，他們只要附加事件監聽器即可！

讓我們重構 `StoryViewer` 的 `index` 屬性，改用 setter，這樣就能輕鬆透過程式碼路徑調度事件：

```
class StoryViewer extends LitElement {
  @state() private _index: number = 0
  get index() {
    return this._index
  }
  set index(value: number) {
    this.children[this._index].dispatchEvent(new CustomEvent('exited'));
    this.children[value].dispatchEvent(new CustomEvent('entered'));
    this._index = value;
  }
}
```

如要完成自動播放功能，我們會在 `StoryCard` 建構函式中新增「entered」和「exited」的事件監聽器，用來播放及暫停影片。

請注意，元件使用者可能會在媒體插槽中提供 `<story-card>` 視訊元素，也可能不會。甚至可能完全不會在媒體廣告空間中提供元素。我們必須小心，不要在 `img` 或空值上呼叫 `play`。

返回 `story-card.ts`，新增以下內容：

```

import { query } from 'lit/decorators.js';

class StoryCard extends LitElement {
  constructor() {
    super();

    this.addEventListener("entered", () => {
      if (this._slottedMedia) {
        this._slottedMedia.currentTime = 0;
        this._slottedMedia.play();
      }
    });

    this.addEventListener("exited", () => {
      if (this._slottedMedia) {
        this._slottedMedia.pause();
      }
    });
  }

  /**
   * The element in the "media" slot, ONLY if it is an
   * HTMLMediaElement, such as <video>.
   */
  private get _slottedMedia(): HTMLMediaElement|null {
    const el = this._mediaSlot && this._mediaSlot.assignedNodes()[0];
    return el instanceof HTMLMediaElement ? el : null;
  }

  /**
   * @query(selector) is shorthand for
   * this.renderRoot.querySelector(selector)
   */
  @query("slot[name=media]")
  private _mediaSlot!: HTMLSlotElement;
}

```

自動播放完畢。✅

8. Tip the Scales

現在我們已具備所有基本功能，接下來要再新增一項功能：甜美的縮放效果。我們再次回到 `StoryViewer` 的 `update` 方法。系統會進行一些數學運算，取得 `scale` 常數中的值。如果子項處於啟用狀態，則為 `1.0`，否則為 `minScale`，並在這兩個值之間插補。

將 `story-viewer.ts` 中 `update` 方法的迴圈變更為：

```
update(changedProperties: PropertyValues) {
  // ...
  const minScale = 0.8;
  Array.from(this.children).forEach((el: Element, i) => {
    const x = (i - this.index) * width + deltaX;

    // Piecewise scale(deltaX), looks like: __/\__
    const u = deltaX / width + (i - this.index);
    const v = -Math.abs(u * (1 - minScale)) + 1;
    const scale = Math.max(v, minScale);
    // Include the scale transform
    (el as HTMLElement).style.transform = `translate3d(${x}px,0,0) scale(${scale})`;
  });
  // ...
}
```

就這麼簡單！這篇文章涵蓋許多內容，包括部分 `LitElement` 和 `lit-html` 功能、HTML slot 元素和手勢控制。

如要查看這個元件的完整版本，請前往：<https://github.com/PolymerLabs/story-viewer>。
